

Day 22 : Debugging and DevTools

1. Introduction to Flutter Debugging

Debugging is the process of finding and fixing errors (bugs) in an application. Flutter provides several tools and techniques to debug applications efficiently.

2. Basic Debugging Techniques

A. Basic Debugging Techniques

The `print()` method displays messages in the debug console.

```
void main() {  
  int a = 10;  
  int b = 20;  
  
  print("Value of A: $a");  
  print("Value of B: $b");  
  
  int sum = a + b;  
  print("Sum: $sum");  
}
```

Output :

Value of A: 10

Value of B: 20

Sum: 30

B . Using `debugPrint()`

`debugPrint()` prints long messages without truncating them.

```
import 'package:flutter/material.dart';  
  
void main() {  
  String message = "Flutter debugging is very important.";  
  
  debugPrint(message);  
}
```

Difference between `print()` and `debugPrint()`

<code>print()</code>	<code>debugPrint()</code>
Simple logging	Better logging
Large text may be truncated	Handles large text efficiently
Less optimized	Optimized for Flutter

3. Flutter Inspector in IDE

Flutter Inspector helps inspect widgets and UI hierarchy.

Features

- View widget tree.
- Inspect UI elements.
- Check padding and alignment.
- Detect layout problems.

Steps to Open Inspector

In VS Code

1. Run Flutter application.
2. Press **Ctrl + Shift + P**.
3. Search **Flutter: Open DevTools**.
4. Select **Flutter Inspector**.

In Android Studio

1. Run application.
2. Open **View → Tool Windows → Flutter Inspector**.

4. Using Breakpoints

Breakpoints pause application execution at a specific line.

Steps

1. Open Dart file.
2. Click left margin beside line number.
3. Red dot appears.
4. Run app in Debug mode.

5. App stops at breakpoint.

Example

```
void main() {  
  int x = 5;  
  int y = 10;  
  
  // Breakpoint here  
  int result = x + y;  
  
  print(result);  
}
```

5. Flutter DevTools

Flutter DevTools is a suite of performance and debugging tools.

Installation

Step 1

Install Flutter SDK.

Step 2

Run:

```
flutter pub global activate devtools
```

Step 3

Launch DevTools:

```
flutter pub global run devtools
```

6. Widget Inspector

Widget Inspector visualizes widget hierarchy.

Features

- Select widgets.
- Analyze layout.
- View constraints.
- Inspect padding and margins.

```
MaterialApp  
├─ Scaffold  
│   └─ Column  
│       ├── Text  
│       ├── TextField  
│       └─ ElevatedButton
```

7. Timeline View

Timeline view measures application performance.

Shows

- Frame rendering time.
- CPU usage.
- GPU rendering.
- UI jank.

Steps

1. Open DevTools.
2. Select **Performance** tab.
3. Click **Record**.
4. Interact with app.
5. Stop recording.

8. Memory Profiler

Memory Profiler tracks memory usage.

Helps identify

- Memory leaks.
- Excessive memory usage.
- Object allocations.

Example Problem

```
class MyPageState extends State<MyPage> {  
  
    final TextEditingController controller =  
        TextEditingController();  
  
    @override  
    void dispose() {  
        controller.dispose();  
        super.dispose();  
    }  
  
    @override  
    Widget build(BuildContext context) {  
        return TextField(  
            controller: controller,  
        );  
    }  
}
```

9. Network Inspector

Used to monitor API requests.

Displays

- Request URL.
- Headers.
- Response data.
- Request time.

```
import 'package:http/http.dart' as http;

Future<void> fetchData() async {
  var response = await http.get(
    Uri.parse("https://jsonplaceholder.typicode.com/posts")
  );

  print(response.body);
}
```

10. Logging View

Displays logs generated by application.

Example

```
debugPrint("User Login Successful");
debugPrint("Fetching API Data");
```

11. Debug Common Issues

A) Layout Overflow Error

```
SingleChildScrollView(
  child: Column(
    children: [
      Text("Very long text")
    ],
  ),
)
```

If we use only column that followed by children then it gives error like of overflowed pixels.

B) State Not Updating

```
setState(() {  
  count++;  
});
```

If we try to initialize count and then used elevated button to increase the count it will not refresh the UI.

C) Performance Issues

```
Widget build(BuildContext context) {  
  
  List data = [];  
  
  for(int i=0;i<100000;i++) {  
    data.add(i);  
  }  
  
  return Text("Flutter");  
}
```

D) Memory Leaks

```
@override  
void dispose() {  
  controller.dispose();  
  super.dispose();  
}
```

E) Network Errors

```
try {  
  var response = await http.get(url);  
  
  if(response.statusCode == 200) {  
    print(response.body);  
  }  
  
} catch(e) {  
  print(e);  
}
```

12. Performance Optimization

A) Identify Jank

Jank means dropped frames causing UI lag.

Use:

DevTools → Performance Tab

Look for red bars indicating slow frames.

B) Optimize Widget Rebuilds

```
Consumer<CounterProvider>(  
  builder: (context, provider, child) {  
    return Text(  
      "${provider.count}"  
    );  
  },  
);
```

C) Use const Constructors

This is imp to use in flutter application whenever the widgets requires it or it is automatically suggested by the vs code or in android studio.

```
const Text("Hello")
```

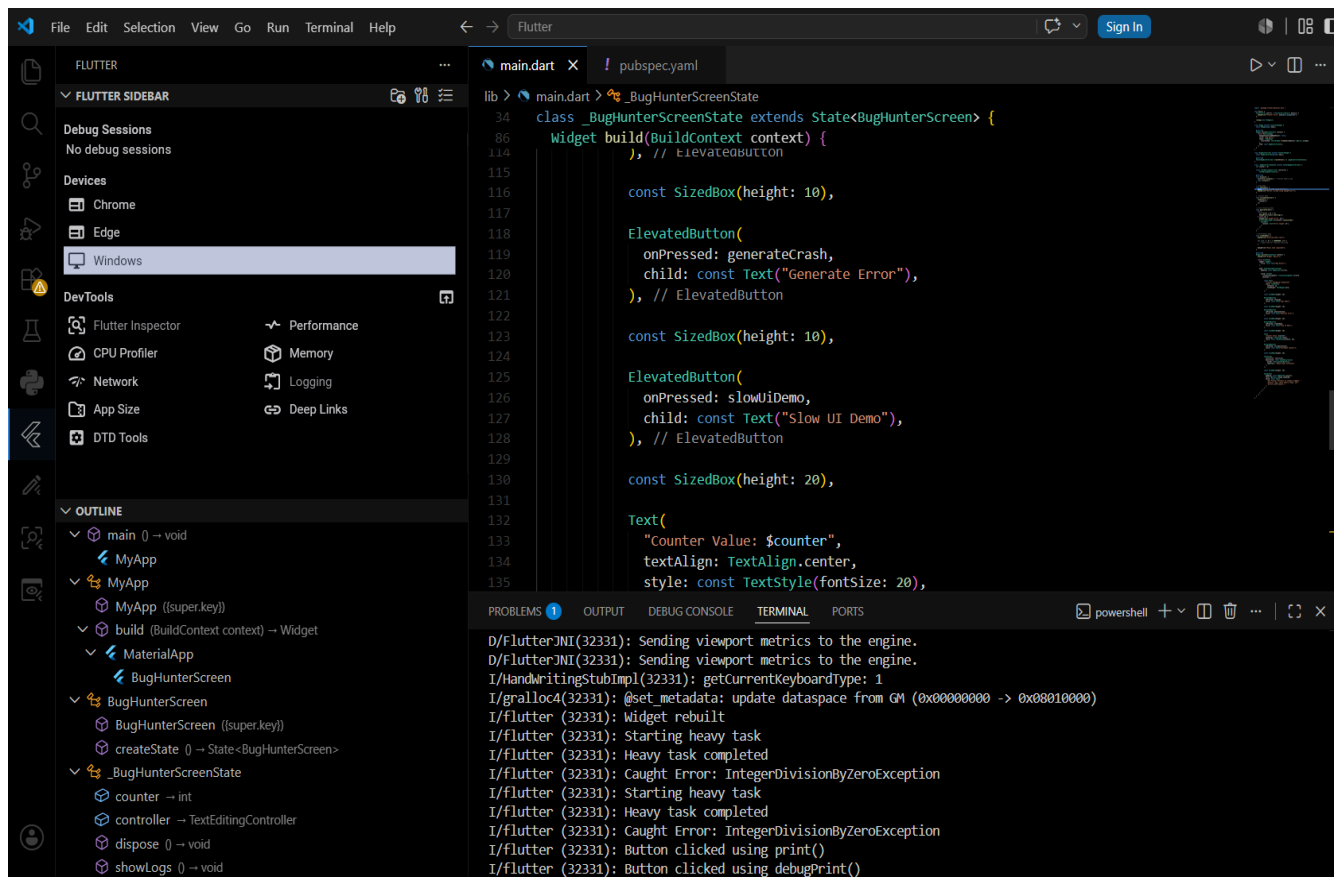
D) Avoid Expensive Operations in build()

```
@override  
void initState() {  
  super.initState();  
  calculateData();  
}
```

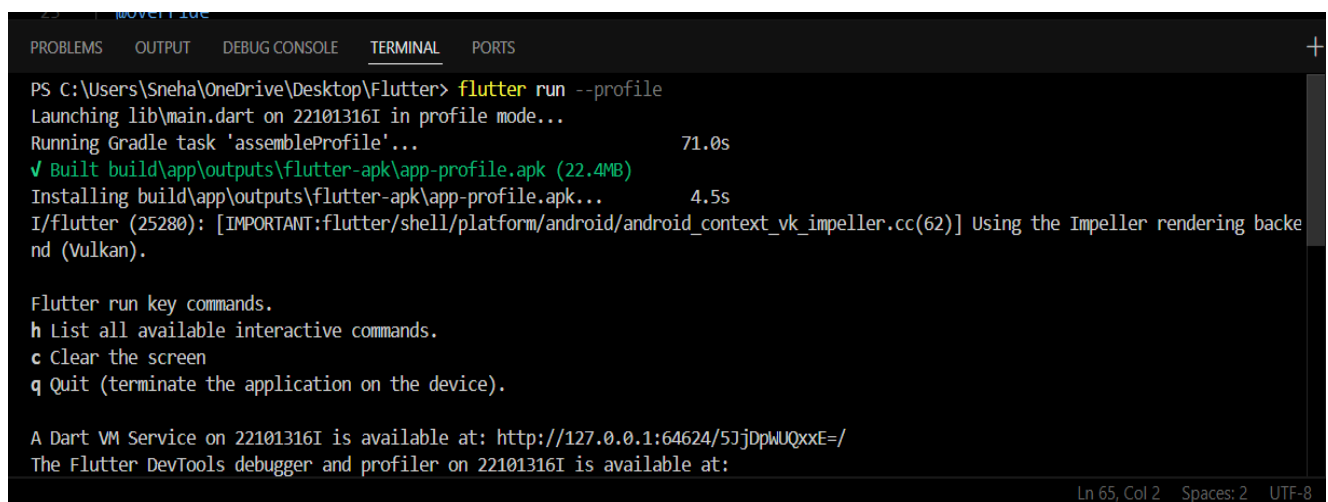
14. Error Handling

A) Try-Catch Blocks

```
try {  
  
  int result = 10 ~/ 0;  
  
} catch(e) {  
  
  print("Error: $e");  
  
}
```



Terminal Run app in performance mode :



12:47

Bluetooth, Wi-Fi, 5G, Cellular, and Battery status icons.

Random Images

